A GUIDE TO THE VERIFIED METAMATH PROJECT

Six nonhalting proofs in bounded arithmetic

From a four-configuration cycle to Finned #3

This document explains how $I\Delta_0$ proves nonhalting for six particular blank-start Turing machines: a cycler, a translated cycler, a counter, a bouncer, a four-state fractal, and Finned #3.

No knowledge of Busy Beaver deciders is assumed. The central objects are ordinary machine instructions, arithmetic descriptions of tapes, and properties preserved by every step. The specific induction formulas are written out, including their bounds.

Scope. A mathematical exposition of the project proofs, with explicit definitions, complete phase descriptions and worked deductions. It does not reproduce every foundational natural-deduction line of the Metamath database. Supporting arithmetic lemmas are identified and explained. The accompanying verified database supplies those formal derivations.

Encoding. One two-half-tape encoding and one quotient-and-remainder history format are used throughout. Invariants are proved about computations; they are never added to the definition of a computation.

Contents

1 What a proof of nonhalting needs to accomplish 2

2 The arithmetic available in $I\Delta_0$ 3

3 One arithmetic encoding of all six machines 4

4 Arithmetic lemmas and the first induction instance 5

5 Coding a finite history and inducting over it 6

6 The cycler: a finite invariant 8

7 The translated cycler: repeat a shape, not a number 9

8 The counter: a parity proof of safety 10

9 The bouncer: a product and a remainder 11

10 The four-state fractal: complicated growth, simple safety 13

11 Finned #3: turn matching tape lengths into algebra 14

12 What has been proved, and how to check it 20

1 What a proof of nonhalting needs to accomplish

A Turing machine repeatedly reads one tape cell, writes a bit, moves one cell left or right, and changes state. Its tape initially contains only zeros. A designated instruction leads to a halting state. A nonhalting proof must rule out that instruction at *every* time, not merely in a large finite simulation.

In Busy Beaver work, a *decider* or *filter* is a method that recognizes some machines whose outcomes can be established. It need not settle every machine. For example, a cycler detector can recognize that a configuration has repeated. Determinism then makes the intervening behavior repeat forever. A soundness proof explains why acceptance by such a detector justifies its conclusion.

This document takes a more specific task: prove that six given machines never halt. These individual theorems are not general soundness theorems for every detector bearing a related name. In particular, the counter and fractal proofs below use simple safety properties without formally deriving the machines' complete long-term growth laws.

1.1 The three obligations

A *configuration* contains the current state, scanned bit and tape. An *invariant* is a property $\mathrm{Inv}(c)$ of configurations. The proof has three parts:

1. The blank configuration satisfies Inv.
2. If $\mathrm{Inv}(c)$ holds and c takes one ordinary step to d, then $\mathrm{Inv}(d)$ holds.
3. No halting configuration satisfies Inv.

Induction over time turns these facts into nonhalting. The first difficulty is finding the invariant. The second, in a weak theory, is expressing it so that the theory's allowed induction really applies.

An invariant may include configurations that never occur. It is a safe *overestimate* of the possible behavior. We do not need to characterize exactly which configurations are reachable.

1.2 What the formal conclusion says

For each machine the project proves that there is no blank-start halting history in the arithmetic history format defined in Section 5. The definition records only ordinary steps. In the standard natural numbers, every genuine finite run has such a code, so this excludes an actual halt. Internally the proof starts with a hypothetical supplied code and refutes it; it does not assume a total history-coding function or total exponentiation. This distinction is part of the theorem's precise meaning in weak arithmetic.

2 The arithmetic available in $I\Delta_0$

The language has $0, 1, +, \cdot, <, =$ and first-order logical operations. Variables range over nonnegative integers in the intended interpretation. Numerals such as 3 abbreviate $1 + 1 + 1$; x^2 abbreviates $x \cdot x$. There is no exponentiation symbol in the formal language.

The project's arithmetic basis consists of the following universal axioms, together with bounded induction. They are displayed so the reader can see what supports the subsequent algebra. Classical first-order logic with equality supplies substitution, case analysis and quantifier rules.

$$\begin{array}{ccc} (x+y)+z = x+(y+z), & x+y = y+x, & x+0 = x, \\ (xy)z = x(yz), & xy = yx, & x \cdot 1 = x, \\ x \cdot 0 = 0, & x(y+z) = xy + xz. & \end{array}$$

The order axioms are

$$\begin{array}{l} \neg(x < x), \qquad (x < y \wedge y < z) \to x < z, \\ x < y \vee x = y \vee y < x, \qquad \neg(x < 0), \\ x < y \to x + z < y + z, \\ (x < y \wedge 0 < z) \to xz < yz, \\ x < y \iff \exists g\,[y = (x + g) + 1]. \end{array}$$

The last axiom makes the order discrete. Its existential quantifier is unbounded: the restriction to bounded formulas applies to the *induction schema*, not to every axiom of the theory.

2.1 Bounded formulas and parameters

A bounded quantifier is an abbreviation:

$$\exists u < t\,\theta \;:=\; \exists u\,(u < t \wedge \theta), \qquad \forall u < t\,\theta \;:=\; \forall u\,(u < t \to \theta).$$

Its bound t must not contain its bound variable u. A Δ_0 formula is built from arithmetic comparisons using logical connectives and bounded quantifiers only. For example

$$\mathrm{Even}(x) := \exists u < x + 1\ (x = 2u), \qquad \mathrm{Odd}(x) := \exists u < x + 1\ (x = 2u + 1)$$

are bounded formulas. So are finite disjunctions of such formulas.

For each bounded formula $\varphi(i; \mathbf{p})$, $I\Delta_0$ allows

$$\big[\varphi(0; \mathbf{p}) \wedge \forall i\,(\varphi(i; \mathbf{p}) \to \varphi(i + 1; \mathbf{p}))\big] \;\to\; \forall i\,\varphi(i; \mathbf{p}). \tag{1}$$

Here i is the induction variable. The tuple $\mathbf{p}$ lists arbitrary other parameters held fixed during that application. The universal quantifiers outside φ in (1) do not make the instance illegal. It is φ itself that must be bounded.

An assertion such as "a configuration occurs after i steps" can hide unbounded existence of a large computation code. We avoid placing such an unbounded existence assertion inside the induction formula. Instead, we fix a proposed finite history code and reason about its bounded rows.

3 One arithmetic encoding of all six machines

A configuration is the quadruple

$$c = (q, L, b, R).$$

The state codes are A=0, B=1, C=2, D=3, E=4, halt Z=5. The scanned bit is b. The integers L, R describe the tape strictly left and right of the head; they do *not* include the scanned cell. On either side, the nearest cell is the least significant binary digit. Unwritten cells, arbitrarily far away, are zero.

For example, the right-hand sequence of bits $1, 0, 1$ beginning immediately beside the head has code $R = 1 + 0 \cdot 2 + 1 \cdot 4 = 5$. The left sequence $0, 1$ beginning immediately beside the head has code $L = 2$. These explanatory place-value descriptions use ordinary binary notation. The formal step definition below needs only multiplication by 2.

3.1 Push and pop as equations

When an instruction writes e and moves *right*, the old scanned cell becomes the nearest cell on the left. The nearest right bit becomes scanned. Thus

$$L' = 2L + e, \qquad R = 2R' + b', \qquad b' = 0 \vee b' = 1. \tag{2}$$

A left move gives

$$R' = 2R + e, \qquad L = 2L' + b', \qquad b' = 0 \vee b' = 1. \tag{3}$$

These equations are an arithmetic specification of a step. On ordinary natural numbers they give an algorithm: push by doubling and adding e; pop by dividing by 2, taking quotient as the remaining half-tape and remainder as the new scanned bit.

For instance, $c = (\mathrm{A}, 2, 0, 5)$ under a write-1/right instruction becomes $(q', 5, 1, 2)$: $5 = 2 \cdot 2 + 1$ is both the pushed left word and the old right word's quotient-and-remainder decomposition. Under a write-0/left instruction, the same c becomes $(q', 1, 0, 10)$.

A transition table gives two instructions per nonhalting state, ordered by scanned bit $0, 1$. The compact instruction `1RB` means "write 1, move right, enter B." An underscore separates state rows. We complete every `---` instruction as `1RZ`. Its final write and movement are recorded. There are no outgoing steps from Z.

For a fixed machine, $\mathrm{Step}(c, d)$ is the finite disjunction of its instruction clauses. Each clause states the source state and bit, the next state, and equations (2) or (3). It is quantifier-free. No parity condition or invariant is built into it.

3.2 Why equations suffice inside the theory

Suppose a step pops an old half-tape $2u$. Its clause gives $2u = 2R' + b'$ and $b' < 2$. Uniqueness of quotient and remainder implies $b' = 0$ and $R' = u$. This is how the theory learns what bit is read; it is not consulting a hidden simulator or assuming an intended tape pattern. The same argument forces bit 1 when the old half is $2u + 1$.

4 Arithmetic lemmas and the first induction instance

Most local step proofs use elementary consequences of the arithmetic axioms: additive cancellation, cancellation of a positive factor, nonnegativity, and uniqueness of quotient and remainder. None of those requires induction in this project.

For uniqueness, if

$$mq + r = mq' + r', \qquad r < m, \quad r' < m,$$

then $q = q'$ and $r = r'$. To see the central argument, assume $q < q'$. Discreteness gives $q' = q+g+1$. Cancellation then makes $r = m(g+1)+r' \geq m$, contradicting $r < m$. The case $q' < q$ is symmetric; trichotomy leaves equality. These inequalities can be derived from the order and semiring axioms. A zero-or-one bit disjunction gives the needed remainder bounds without any division-existence assertion.

4.1 Division existence: its exact induction formula

The bouncer and Finned #3 proofs also use *existence* of division. Fix a parameter $m > 0$ and induct on i using

$$D(i; m) := \exists q < i + 1 \; \exists r < m \; [i = mq + r]. \tag{4}$$

This is bounded: both witnesses have explicit term bounds. The full instance is (1) with $\varphi = D$ and parameter m.

At $i = 0$, choose $q = r = 0$. For the successor, eliminate the old witnesses. Because $r < m$, discreteness gives $r + 1 < m$ or $r + 1 = m$.

- In the first case use the same q and new remainder $r + 1$.
- In the second use quotient $q + 1$ and remainder 0.

The old bound $q < i + 1$ ensures $q + 1 < i + 2$; the unchanged quotient also fits the new bound. This proves division for any positive divisor, including 2 and 3. Reusing this proved theorem does not create a new induction construction each time.

4.2 A bounded substitute for writing "$p = 2^k$"

Define

$$\begin{aligned} \mathrm{Div}(d, p) &:= \exists e < p + 1 \; (de = p), \\ \mathrm{Pow2}(p) &:= 0 < p \; \wedge \; \forall d < p + 1 \big(\mathrm{Div}(d, p) \to [d = 1 \vee \mathrm{Even}(d)]\big). \end{aligned}$$

In the standard natural numbers this describes the powers of two: every positive divisor is either 1 or even. We use the *formula* and its proved consequences, not an exponentiation function.

The project proves Pow2(1), closure under doubling, closure under taking positive divisors, and

$$\mathrm{Pow2}(p) \to \big[p = 1 \vee \exists h < p + 1 \; (p = 2h \wedge \mathrm{Pow2}(h))\big].$$

An odd Pow2 number must equal 1. For the last claim, split p by the displayed halving result: its even alternative contradicts oddness by remainder uniqueness. A divisor of a Pow2 number is again Pow2 because its divisors are divisors of the original number. Doubling closure uses parity division of a divisor and its cofactor; it reuses (4). None of these assertions says that 2^k exists for every k.

5 Coding a finite history and inducting over it

A finite list of tuples cannot be used as a primitive object of our first-order arithmetic language. We therefore describe it by numbers. Fix parameters $H, n, o, B_0, C_0, \ldots, B_3, C_3$. The intended final time is n; o is an index offset, and H bounds row coordinates and quotients. Coordinate j has the bounded decoding relation

$$\beta_j(k, v) := \exists u < H \left[B_j = u\big(1 + (k+1)C_j\big) + v \ \wedge \ v < 1 + (k+1)C_j \right]. \tag{5}$$

This says that v is a remainder of B_j modulo the indicated positive number. For example, with $B_j = 17, C_j = 2$, the moduli at $k = 0, 1, 2$ are $3, 5, 7$ and the remainders are $2, 2, 3$. Arbitrary choices need not code a valid run: the following predicates impose that requirement.

For $c = (q, L, b, R)$ let

$$\mathrm{At}(i, c) := \beta_0(o + i, q) \wedge \beta_1(o + i, L) \wedge \beta_2(o + i, b) \wedge \beta_3(o + i, R).$$

The notation $\forall c < H$ or $\exists c < H$ abbreviates four separate bounded quantifiers, one for each coordinate. It is not a new tuple sort. Define

$$\begin{aligned} T &:= \forall i < n + 1 \ \exists c < H \ \ \mathrm{At}(i, c), \\ S &:= \forall i < n \ \forall c, d < H \big[(\mathrm{At}(i, c) \wedge \mathrm{At}(i + 1, d)) \to \mathrm{Step}(c, d) \big], \\ B &:= \forall c < H \ [\mathrm{At}(0, c) \to c = (\mathrm{A}, 0, 0, 0)]. \end{aligned}$$

Here T means that each needed row is present, S imposes ordinary steps, and B imposes blank initial data. Tuple equality means four equalities. Neither T, S, B mentions the invariant.

The formal halting statement for machine M is the existential closure of

$$T \wedge S \wedge B \wedge c_f < H \wedge \mathrm{At}(n, c_f) \wedge q_f = 5 \tag{6}$$

over H, n, o, the eight code parameters, and c_f. Here $c_f < H$ means all four coordinates are below H. Nonhalting is its negation. Each verified endpoint theorem refutes this conjunction for arbitrary free parameters; universal generalization gives that negation.

5.1 Why the encoding does not assume the answer

The history definition supplies states, tape halves and bits at successive times, with ordinary transitions between them. It supplies no parity witnesses, phase parameters or invariant certificates. Those are *constructed in the proof*. A hypothetical run that violates the invariant is initially allowed by the history definition and is then shown impossible from the axioms.

There is a useful metatheoretic observation: any ordinary finite list of natural numbers can be represented by a suitable remainder code (choose compatible moduli and use the Chinese remainder theorem). H can then be chosen large enough. This explains the standard interpretation of the definition; a general coding-existence theorem inside $I\Delta_0$ is not used in these six proofs.

5.2 The shared history induction, fully specified

Fix all the history parameters and assume T, S and that every initial row satisfies Inv_H. For the first five machines the invariant does not depend on H; for Finned #3 it does. Apply bounded induction to

$$\boxed{\begin{aligned}\Phi_M(i) := i < n+1 \ \rightarrow \\ \forall q < H \ \forall L < H \ \forall b < H \ \forall R < H \\ [\mathrm{At}(i,(q,L,b,R)) \rightarrow \mathrm{Inv}_{M,H}(q,L,b,R)].\end{aligned}} \tag{7}$$

This is a different concrete instance for each machine: substitute its explicit invariant below. All the other displayed symbols are expanded by their definitions. The parameters are H, n, o, B_j, C_j, not induction variables.

Base. Given a row at time 0, blank initial data identify its coordinates. Exhibit the initial invariant witnesses. This proves $\Phi_M(0)$, including the vacuous case when its antecedent is false.

Successor. Assume $\Phi_M(i)$, and suppose $i+1 < n+1$. Then $i < n$ and $i < n+1$. Take an arbitrary supplied row $d < H$ at time $i+1$. By T, choose a row $c < H$ at time i. Apply the induction hypothesis to get $\mathrm{Inv}_{M,H}(c)$. By S we have $\mathrm{Step}(c,d)$. The local preservation theorem gives $\mathrm{Inv}_{M,H}(d)$. Discharge the chosen row and universally quantify the coordinates of d. If $i+1 < n+1$ is false, the successor implication is vacuous. Thus the proof establishes the successor for *every* i, as the induction schema requires.

Conclusion. Instantiate the resulting $\forall i \, \Phi_M(i)$ at n. Since $n < n+1$, the supplied final row satisfies the invariant. Its state is therefore not 5, contradicting (6).

5.3 Why this is allowed, and a tempting alternative is not

Every quantifier in (7) is bounded. This includes those hidden inside At, parity predicates, Pow2, and the Finned phase certificates. The formula may be long, have implications and disjunctions, and contain many parameters. None of that disqualifies bounded induction.

In contrast, a formula of the form "there exists an unrestricted code of the entire computation through time i" begins with unbounded existential quantifiers. It is not automatically Δ_0. Nor can we write 2^i as an available term and assume it is total. Fixing a supplied history with coordinate bound H avoids both issues.

There is no fixed numerical time cutoff. The induction works for arbitrary n and arbitrary admissible codes. Nor is H fixed once for the machine: it is an arbitrary parameter for the particular hypothetical finite history.

6 The cycler: a finite invariant

`1RB0RB_0LA---`

State	Read 0	Read 1
A	1RB	0RB
B	0LA	1RZ

Only B1 halts. Starting from blank, the following four configurations repeat. We list (q, L, b, R), with state letters standing for their codes.

$$c_0 = (\mathrm{A}, 0, 0, 0), \quad c_1 = (\mathrm{B}, 1, 0, 0), \quad c_2 = (\mathrm{A}, 0, 1, 0), \quad c_3 = (\mathrm{B}, 0, 0, 0).$$

Define the quantifier-free formula

$$\mathrm{Inv}_{\mathrm{cyc}}(c) := (c = c_0) \vee (c = c_1) \vee (c = c_2) \vee (c = c_3).$$

This explicitly finite set is the simplest invariant in the document.

6.1 Checking all possible successors

Source	Instruction	Arithmetic forcing the target
c_0	A0	$L' = 1$, $0 = 2R' + b'$, so $R' = b' = 0$: c_1
c_1	B0	$R' = 0$, $1 = 2L' + b'$, so $L' = 0, b' = 1$: c_2
c_2	A1	$L' = 0$, $0 = 2R' + b'$, so $R' = b' = 0$: c_3
c_3	B0	$R' = 0$, $0 = 2L' + b'$, so $L' = b' = 0$: c_0

Every pop uses remainder uniqueness with remainder below 2. The full Step disjunction also contains instructions with the wrong source state or scanned bit. Those alternatives contradict the source equalities and are eliminated by ordinary case analysis. In particular, the halting B1 branch is checked and excluded, rather than removed from the transition definition.

Blank is c_0. Every invariant clause has state A or B, and none has state Z. Substitute this displayed finite disjunction into (7); that is the specific induction instance for this machine. One bounded history induction completes nonhalting. The local closure proof requires no induction, including no division *existence*: the step already supplies a quotient and a bit.

> The familiar algorithmic explanation is "the same configuration repeats." The arithmetic proof expresses the same reason by a finite disjunction, checks its closure, and propagates it along an arbitrary proposed history. It does not need a primitive function returning the configuration at time i.

7 The translated cycler: repeat a shape, not a number

`1RB1RA_1LA---`

State	Read 0	Read 1
A	1RB	1RA
B	1LA	1RZ

A translated cycler repeats a pattern while its position changes. Consequently its numerical tape codes need not repeat. Our proof does not explicitly keep a head-position coordinate: L, R are always relative to the current head.

For this machine use the three-region invariant

$$\begin{aligned}\mathrm{Inv}_{\mathrm{tr}}(q, L, b, R) := (q = \mathrm{A} \wedge R = 0) \\ \vee (q = \mathrm{A} \wedge b = 1 \wedge R = 1) \\ \vee (q = \mathrm{B} \wedge b = 0 \wedge R = 0 \wedge \mathrm{Odd}(L)).\end{aligned}$$

In the first clause b is unrestricted; an actual step still requires $b = 0$ or 1. In the B clause, the nearest left bit is 1, as expressed by the bounded witness for $\mathrm{Odd}(L)$.

7.1 Preservation of the three regions

1. At A with $R = 0, b = 0$, instruction A0 pushes 1 on the left: $L' = 2L + 1$. The right pop of zero gives $b' = R' = 0$. The new left half is odd, witnessed by the old L, so the B clause holds. Its bound is explicit: $L < L' + 1$ because $L' + 1 = 2L + 2$.
2. At A with $R = 0, b = 1$, instruction A1 moves right, and the same pop gives $R' = 0$. The first A clause holds again.
3. At A with $b = 1, R = 1$, instruction A1 gives $R' = 0, b' = 1$. Again the first clause holds.
4. In the B clause, eliminate its witness $L = 2u + 1$. B0 writes 1 and moves left. Therefore $R' = 2R + 1 = 1$ and $L = 2L' + b'$. Remainder uniqueness gives $b' = 1, L' = u$. The second A clause holds. B1 is impossible because this source clause already has $b = 0$.

These exhaust the possible steps from the invariant.

7.2 What the changing tape codes look like

The first few ordinary rows are

$$(\mathrm{A}, 0, 0, 0),\ (\mathrm{B}, 1, 0, 0),\ (\mathrm{A}, 0, 1, 1),\ (\mathrm{A}, 1, 1, 0),\ (\mathrm{A}, 3, 0, 0),\ (\mathrm{B}, 7, 0, 0).$$

The values grow, but each row belongs to one of the three regions. The proof needs only the zero/one values on the right and oddness at B, not a formula predicting every growing value of L.

The blank tuple satisfies the first clause. All clauses exclude halt. The induction instance is exactly (7) with $\mathrm{Inv}_{M,H} = \mathrm{Inv}_{\mathrm{tr}}$. Its hidden oddness quantifier is bounded by $L + 1$, so this is legal Δ_0 induction. The local proof has no induction dependency; the endpoint proof has the one history induction.

8 The counter: a parity proof of safety

`1RB---_1LC1RB_0RA0LC`

State	Read 0	Read 1
A	1RB	1RZ
B	1LC	1RB
C	0RA	0LC

The computation has counter-like growth. To establish nonhalting we do not need to interpret the entire counter or prove that an arbitrary counter value has an encoding. The only dangerous instruction is A1. We prove that every visit to A scans 0.

The invariant is

$$\begin{aligned}\mathrm{Inv}_{\mathrm{ctr}}(q, L, b, R) := (q = \mathrm{A} \wedge b = 0) \\ \vee (q = \mathrm{B} \wedge \mathrm{Odd}(L)) \\ \vee (q = \mathrm{C} \wedge [b = 1 \vee \mathrm{Even}(R)]).\end{aligned}$$

The C clause has a useful conditional meaning: *if C scans 0*, then its right half must be even. It does not require evenness when C scans 1.

8.1 All instruction cases

Instruction	Why the target is safe
A0	Pushes 1 on the left, making $L' = 2L + 1$ odd; target B.
A1	Impossible: the A clause gives scanned bit 0.
B0	Write 1 and move left. Since L is odd, the popped bit is 1; target C satisfies its first alternative.
B1	Pushes 1 on the left, retaining oddness; target B.
C1	Write 0 and move left, giving $R' = 2R$ even. Whatever bit is popped, the C clause holds.
C0	The source C clause forces $\mathrm{Even}(R)$. A right pop of even R reads 0, so the target A scans 0.

For example, in the last case eliminate a source witness $R = 2u$. The transition gives $R = 2R' + b'$ with $b' < 2$. Comparing these equations forces $b' = 0$. In C1, the old R witnesses evenness of R'; its required bound $R < R' + 1$ follows from $R' = 2R$ with additive gap R.

Blank satisfies the A clause. The invariant permits only states A,B,C, so cannot hold at halt. Substitute $\mathrm{Inv}_{\mathrm{ctr}}$ into (7). Both parity predicates are bounded, and the resulting instance proves nonhalting.

> There is a difference between proving a machine safe and deriving a complete mathematical description of its computation. Here parity is enough to exclude the one halting instruction. The proof does not assert that all counter deciders are sound, or that every counter machine has this invariant.

9 The bouncer: a product and a remainder

`1RB0LC_0LC1RA_---1LA`

State	Read 0	Read 1
A	1RB	0LC
B	0LC	1RA
C	1RZ	1LA

A bouncer repeatedly travels across and modifies a tape region. To justify this example, the project uses a numerical invariant maintained at *every step*, rather than assuming that a whole sweep is atomic. Its product changes in a small, controlled way during those steps.

Write

$$P_A = (L+1)(3R+1), \qquad P_{BC} = (L+1)(3R+2),$$

and define a bounded remainder predicate

$$\mathrm{Res}_j(P) := \exists k < P+1 \ (P = 3k + j), \qquad j = 1, 2.$$

The invariant is the disjunction of these five clauses:

State/read	Product P	Required predicates	Extra condition
A0	P_A	$\mathrm{Pow2}(P),\ \mathrm{Res}_1(P)$	none
A1	P_A	$\mathrm{Pow2}(P),\ \mathrm{Res}_2(P)$	none
B0	P_{BC}	$\mathrm{Pow2}(P),\ \mathrm{Res}_1(P)$	$R = 0$
B1	P_{BC}	$\mathrm{Pow2}(P),\ \mathrm{Res}_1(P)$	none
C1	P_{BC}	$\mathrm{Pow2}(P),\ \mathrm{Res}_2(P)$	none

A row means the conjunction of the state/read equalities and the listed conditions. The table has no C0 clause. This is an explicit bounded formula: P_A, P_{BC} are terms and both $\mathrm{Pow2}, \mathrm{Res}_j$ expand into bounded quantifiers.

9.1 Why both parts of the invariant are useful

The power condition restricts factors. If a product is Pow2, each positive factor is Pow2. An *odd* factor must consequently be 1. The remainder condition then distinguishes possibilities that the power condition alone would allow. For instance, at A1, if a left pop read 0, then $L = 2L'$ makes $L+1$ odd. It must be 1, giving $L = 0$. The product then becomes $3R+1$, whose remainder modulo 3 is 1, contradicting the A1 condition $\mathrm{Res}_2(P_A)$. Thus that pop must read 1. This prevents A1 from entering the dangerous C0 configuration.

9.2 A complete one-step update table

Let P be the source product and P' the product for the indicated target clause. The ordinary instruction and its pop bit give:

Source	New bit	Target	Product relation
A0	0	B0	$P' = 4P$; also $R' = 0$
A0	1	B1	$P' = P$
A1	1	C1	$P' = P$
B0	1	C1	$P = 2P'$
B1	0	A0	$P' = P$
C1	0	A0	$P' = 2P$
C1	1	A1	$P' = P$

Every product relation is verified by substitution into the push/pop equations. We also have to exclude the other pop bits, not just check this table of acceptable ones.

At B0, the extra $R = 0$ is essential. If its left pop read 0, then $L + 1$ would be an odd Pow2 factor and equal 1. Consequently $P_{BC} = 2$, contradicting remainder 1 modulo 3. At B1, a right pop of 1 would make $3R + 2 = 6R' + 5 = 2(3R' + 2) + 1$ an odd Pow2 factor. It would have to equal 1, impossible because $3R' + 2 > 0$. The impossible A1 pop was explained above. At C1, a left pop of 0 also makes $L + 1$ an odd Pow2 factor, hence $L = L' = 0$; with this equality the product update is $P' = 2P$. C0 is excluded by the source invariant itself.

For the first table row, the right pop $b' = 0$ makes $3R + 1 = 6R' + 1$ an odd Pow2 factor. It equals 1, so $R' = R = 0$. Direct substitution then gives $P' = 4P$ and the special B0 clause. This also explains why that extra clause condition is preserved.

9.3 Preserving the arithmetic predicates

When $P' = P$, both predicates transfer by equality. When $P' = 2P$ or $4P$, doubling closure supplies Pow2. When $P = 2P'$, divisor closure supplies Pow2 of P'. The remainder changes are

$$P \equiv 2 \pmod 3 \Rightarrow 2P \equiv 1 \pmod 3, \quad P \equiv 1 \pmod 3 \Rightarrow 4P \equiv 1 \pmod 3,$$

$$P = 2P', \ P \equiv 1 \pmod 3 \Rightarrow P' \equiv 2 \pmod 3.$$

Here congruences are explanatory notation for the bounded Res_j predicates. Their witnesses and bounds are explicitly constructed in the formal proof. The last implication uses division of P' by 3, then eliminates its other two possible remainders.

9.4 Base, induction and endpoint

At blank, the A0 product is 1. We know Pow2(1) and may choose $k = 0 < 2$ for $1 = 3k + 1$. All ordinary successors preserve a table clause, and none permits halt. Use (7) with this five-clause invariant. Together with division induction (4), this accounts for the two bounded-induction nodes in the endpoint proof's dependency closure. There is no induction on an unbounded sweep length hidden in the local step table.

10 The four-state fractal: complicated growth, simple safety

`1RB0LA_1RC---_0RD0RC_1LD0LA`

State	Read 0	Read 1
A	1RB	0LA
B	1RC	1RZ
C	0RD	0RC
D	1LD	0LA

The machine's self-similar behavior motivates its name, but the verified nonhalting proof does not require a theorem about that recursion. The invariant is surprisingly small:

$$\mathrm{Inv}_{\mathrm{frac}}(q, L, b, R) := (q = \mathrm{A} \wedge \mathrm{Even}(R)) \vee (q = \mathrm{B} \wedge b = 0) \vee q = \mathrm{C} \vee q = \mathrm{D}.$$

The only state-dependent restrictions concern A and B. In particular, the invariant is not asserting that arbitrary C or D configurations are reachable; it only allows them as safe overestimates.

Instruction	Preservation argument
A0	The old R is even. Its right pop reads 0, giving B0.
A1	Write 0, move left: $R' = 2R$, so the new A clause holds.
B0	Target C, which is unrestricted by the invariant.
B1	Impossible: the source B clause requires $b = 0$.
C0	Target D.
C1	Target C.
D0	Target D.
D1	Write 0, move left: $R' = 2R$, so target A has even right half.

Every line is a consequence of one ordinary instruction. For A0, eliminate $R = 2u$ and use $R = 2R' + b'$ to obtain $b' = 0$. For A1 and D1, the old R is the bounded witness for evenness of R'; $R < R' + 1$ follows directly from $R' = 2R$.

Blank satisfies the A clause using witness $u = 0 < 1$. Every clause has state at most D, and therefore excludes Z. With this exact invariant, (7) is the sole induction instance in the nonhalting proof's dependency closure. Evenness has a bounded existential quantifier, so the instance is legal even though it is not quantifier-free.

> The name "fractal" describes behavior; it does not determine the logical strength of the shortest safety argument. This example does not show that detailed fractal growth laws are easy to formalize, nor that $I\Delta_0$ proves every theorem about fractal machines. It shows that one particular machine has an elementary invariant excluding halt.

11 Finned #3: turn matching tape lengths into algebra

`1RB1RE_1LC1RB_0RA0LD_1LB1LD_---0RA`

State	Read 0	Read 1
A	1RB	1RE
B	1LC	1RB
C	0RA	0LD
D	1LB	1LD
E	1RZ	0RA

The dangerous instruction is E0. This proof does more arithmetic than the earlier examples because its recurring tape pattern relates two unbounded lengths. The safety invariant tracks that relationship during all the intervening sweeps.

11.1 Start with a visible boundary configuration

At certain A0 configurations the left half, read nearest first, is $0, 1, 0, 1, \ldots$ with n pairs, and the right half is n ones. In ordinary numerical notation their values are

$$L = \frac{2(4^n - 1)}{3}, \qquad R = 2^n - 1.$$

These expressions explain the pattern *outside* the formal language. Eliminating n gives the polynomial relation

$$3L + 2 = 2(R + 1)^2. \tag{8}$$

For example:

n	L	R	$3L + 2 = 2(R + 1)^2$
0	0	0	2
1	2	1	8
2	10	3	32
3	42	7	128

This is the main conceptual move. Instead of quantifying a tape length and constructing powers $2^n, 4^n$, the invariant uses products and a Pow2 scale already tied to the tape coordinates.

Introduce $p = R + 1$ with Pow2(p). The A0 boundary clause becomes

$$q = \mathrm{A}, \quad b = 0, \quad 3L + 2 = 2p^2, \quad R + 1 = p.$$

This is not yet a full invariant: one step leaves the A0 form. We need additional clauses covering the rest of the computation.

11.2 The complete ten-phase invariant

Fix the coordinate bound H of a hypothetical history and put

$$U = 16(H + 1).$$

For each table row, existentially quantify *every listed parameter* below U and require Pow2 of every such parameter. Conjoin the state/read equalities and the displayed equations. The disjunction of these ten bounded clauses is $\mathrm{Inv}_{\mathrm{fin,H}}$.

Phase	Scales	Equations
A0	p	$3L + 2 = 2p^2, \quad R + 1 = p$
A1	s, w	$3L + 2 = 2s^2, \quad R + 1 + 2w^2 = 8sw^3$
B1	p, w, z, v	$3(L + 1) = w(2p^2 + 1), \quad R + 1 + z = 2zv, \quad wz = pv^2$
B0-work	p, v	$3(L + 1) = 2pv^2(2p^2 + 1), \quad R + 1 = v$
B0-final	w	$L = 0, \quad R + 1 + 2w^2 = 8w^3$
C1	p, v	$3(L + 1) = pv^2(2p^2 + 1), \quad R + 1 = 2v$
C0-final	w	$L = 0, \quad R + 1 + 4w^2 = 16w^3$
D1	s, w, u, v	$3(L + 1) = w(2s^2 + 1), \quad R + 1 + u = 2uv, \quad 8uw = sv^2$
D0	s, u, v	$3L + 1 = s^2, \quad R + 1 + u = 2uv, \quad 4u = sv^2$
E1	s, w	$3L + 1 = s^2, \quad R + 1 + w^2 = 2sw^3$

"Work" and "final" distinguish two arithmetic descriptions of the same state/read pair. They are not extra machine states. A parameter name is local to its clause: it is a proof witness, not a tape annotation or a persistent variable stored by the machine.

For example, the E1 clause expands to

$$q = 4 \wedge b = 1 \wedge \exists s < U \; \exists w < U \, [\mathrm{Pow2}(s) \wedge \mathrm{Pow2}(w) \wedge 3L + 1 = s^2 \wedge R + 1 + w^2 = 2sw^3].$$

Expanding Pow2 yields only bounded quantifiers. Every table row is therefore Δ_0. The global invariant never assumes that such witnesses are supplied with a history; the proof creates them.

The parameters encode the changing scales during left and right scans. Halving one scale and doubling another records a head movement across a repeated block. The product equations keep the different scans synchronized. Their roles are made precise by the following complete update table.

11.3 Every phase has a verified ordinary successor

Parameters on the right refer to those on the left unless explicitly updated. A displayed h is supplied by the bounded Pow2-halving lemma; there is no division function in the arithmetic language.

Source	Condition	Target certificate
A0(p)	$p = 1$	B0-work$(p, 1)$
A0(p)	$p = 2h$	B1$(p, 2, h, 1)$
A1(s, w)	always	E1$(2s, w)$
B1(p, w, z, v)	$z = 1$	B0-work(p, v)
B1(p, w, z, v)	$z = 2h$	B1$(p, 2w, h, v)$
B0-work(p, v)	always	C1(p, v)
B0-final(w)	always	C0-final(w)
C1(p, v)	$pv^2 = 1$	D0$(p, 1, 2v)$
C1(p, v)	$pv^2 = 2h$	D1$(p, h, 1, 2v)$
C0-final(w)	always	A1$(1, w)$
D1(s, w, u, v)	$w = 1$	D0$(s, 2u, v)$
D1(s, w, u, v)	$w = 2h$	D1$(s, h, 2u, v)$
D0(s, u, v)	$s = 1$, $v = 2h$	B0-final(h)
D0(s, u, v)	$s = 2h$	B1$(h, 1, 2u, v)$
E1(s, w)	$w = 1$	A0(s)
E1(s, w)	$w = 2h$	A1(s, h)

The source state/read pair selects an actual instruction in the machine table. The transition equations, source certificate and parity uniqueness force the new read bit and establish the target equations. The complete formal step disjunction is used: wrong source instructions are excluded by their state or bit equalities. The missing E0 phase is never assumed absent from Step.

The conditions cover every source certificate. In D0 with $s = 1$, the alternative $v = 1$ would give $4u = 1$, contradicting parity; hence the listed half exists. At C1, proving Pow2(pv^2) requires an extra arithmetic induction described below. All other splits use the existing halving theorem directly.

This table is a finite collection of algebraic proof obligations, not an empirical table inferred from a long simulation. Its cases also cover a history that stops in the middle of a scan. Showing only boundary-to-boundary behavior would not establish that safety property.

11.4 Two worked transitions: why E0 is never entered

The only transition into state E is A1, a write-1/right move. Suppose the source has the A1 certificate (s, w):

$$3L + 2 = 2s^2, \qquad R + 1 + 2w^2 = 8sw^3.$$

The ordinary step equations give

$$L' = 2L + 1, \qquad R = 2R' + b', \qquad b' \in \{0, 1\}.$$

If $b' = 0$, substitution in the right equation gives

$$2(R' + w^2) + 1 = 2(4sw^3) + 0.$$

Uniqueness of remainders below 2 would identify 1 with 0. Therefore $b' = 1$. Cancelling a factor 2 in the remaining equation gives

$$R' + 1 + w^2 = 4sw^3 = 2(2s)w^3.$$

On the left,

$$3L' + 1 = 3(2L + 1) + 1 = 2(3L + 2) = 4s^2 = (2s)^2.$$

Together with Pow2$(2s)$ and Pow2(w), these are exactly the E1 certificate $(2s, w)$. Its new witness bounds are justified in the next subsection. This is a direct arithmetic proof that entering E cannot enter its halting read-0 case.

Now start at E1(s, w). Instruction E1 writes 0 and moves right, so

$$L' = 2L, \qquad R = 2R' + b'.$$

If $w = 1$, the right certificate says $R + 2 = 2s$, so R is even and $b' = 0$. Thus $R' + 1 = s$. Also $3L' + 2 = 2(3L + 1) = 2s^2$. We have reached A0$(s)$.

If $w = 2h$, the right certificate becomes

$$R + 1 + 4h^2 = 16sh^3.$$

Thus R is odd, forcing $b' = 1$. Substitute $R = 2R' + 1$ and cancel 2:

$$R' + 1 + 2h^2 = 8sh^3.$$

The same left equation gives $3L' + 2 = 2s^2$. This is A1(s, h). So E1 either finishes this scan or returns to A1 with the scale halved. No separate induction over the scan's length is required for these one-step deductions.

11.5 Why all phase witnesses are bounded

The arithmetic equalities alone would give an invariant with unbounded existential witnesses. That would not automatically be suitable for bounded induction. We must prove that witnesses below $U = 16(H + 1)$ can be maintained whenever the supplied tape coordinates are below H.

The formal bounds use elementary factor estimates. For positive integers u, v, each factor is at most uv. Consequently:

- From $3L + 1 = s^2$ and Pow2(s), we get $s \leq s^2 = 3L + 1 < U$.
- From $3(L + 1) = w(2s^2 + 1)$, positivity gives $w \leq 3(L + 1) < U$.
- From $R + 1 + u = 2uv$ with $u, v > 0$, write $u = a + 1, v = k + 1$. Cancellation gives $R + 1 = u(2k + 1)$, whence $u < R + 2$, and

$$R + 2 = v + (2ak + a + k) + 1,$$

  so also $v < R + 2 < U$.

The predecessor witnesses a, k here are obtained by discreteness from positivity; this is not induction. A general explicit bound $3L + 4 < U$ follows from $L < H$, giving slack for the left-side estimates.

Carried parameters retain their old bounds. Halves inherit bounds from the old parameter. Constants $1, 2$ fit U. For newly doubled parameters, the target equations tie them back to a target tape coordinate and the same factor estimates give the bound. In the A1-to-E1 example, the *new* parameter $2s$ has square $3L' + 1$, and $L' < H$ is supplied by the history row, so $2s < U$.

Thus the assembled local theorem has the precise form

$$\mathrm{Inv}_{\mathrm{fin,H}}(c) \wedge \mathrm{Step}(c, d) \wedge d.L < H \wedge d.R < H \;\rightarrow\; \mathrm{Inv}_{\mathrm{fin,H}}(d).$$

The last two assumptions are already true of every supplied history row. They do not restrict which instructions count as steps. The bound H is fixed only for the hypothetical finite history, not for the whole infinite computation.

11.6 The extra arithmetic induction: multiplying Pow2 scales

The C1 split in the update table needs Pow2(pv^2). Given two Pow2 numbers x, y, the project proves Pow2(xy) by bounded induction with y as a fixed parameter, under assumption Pow2(y). Its exact formula is

$$\boxed{\Psi(i; y) := \forall x < i + 1\ [\mathrm{Pow2}(x) \rightarrow \mathrm{Pow2}(xy)].} \tag{9}$$

At $i = 0$ there is no positive $x < 1$, so the assertion is vacuous. For the successor, old inputs are handled by the induction hypothesis. The new input $x = i + 1$ is either 1, when $xy = y$, or $2h$ with Pow2(h) and $h < x$. The hypothesis supplies Pow2(hy) and doubling closure supplies Pow2($2hy$) = Pow2(xy).

All quantifiers in (9), including those inside Pow2, are bounded. This is ordinary bounded induction implementing a strong-induction argument. It concerns products of *given* scales; it does not produce 2^k for arbitrary k. Apply multiplication closure to v, v, then to p, v^2, to justify the C1 split.

11.7 The final history induction for Finned #3

At blank, choose $p = 1$ for the A0 clause. Its equations are $2 = 2$ and $1 = 1$. The initial row has coordinates below H, which ensures that this witness fits $U = 16(H + 1)$.

The precise induction formula is (7) with $\mathrm{Inv}_{M,H} = \mathrm{Inv}_{\mathrm{fin,H}}$, the full bounded disjunction in Section 11.2. In particular it includes *bounded* existential quantification of every phase parameter and the bounded quantifiers inside each Pow2 predicate. No invariant witness appears in T, S or B.

In the successor argument, eliminate the source phase witnesses below U, apply its row of the update table, use the target coordinate bounds to bound new parameters, and reintroduce the target's bounded existential witnesses. This is how the theory constructs a certificate at every supplied time. It does not assume a sequence of certificates exists as another coded object.

At the final supplied row, one of the ten phase clauses must hold. Each has state A through E and hence state code 0 through 4. The halting row would have code 5, a contradiction. In particular the E clause also forces read bit 1, explaining the machine-level safety reason behind the final state contradiction.

There are three bounded-induction application nodes in this endpoint proof's dependency closure:

1. Division existence with formula $D(i; m)$ in (4).
2. Pow2 multiplication with formula $\Psi(i; y)$ in (9).
3. History propagation with formula $\Phi_{\mathrm{fin}}(i)$ in (7), using the ten-phase invariant.

These are properties of this particular verified proof. They are not lower bounds on the number of induction instances needed by every possible proof, and do not establish independence over any weaker theory.

> The many equations are bookkeeping for intermediate scans. The conceptual spine remains short: encode the matching boundary lengths by a polynomial identity, extend it to every phase, keep all auxiliary scales bounded, and apply bounded induction to an arbitrary proposed halting history.

12 What has been proved, and how to check it

Machine	Local count	Endpoint count	Main safety information
Cycler	0	1	Four configurations
Translated cycler	0	1	Three tape regions
Counter	0	1	Parity at A, B, C
Bouncer	1	2	Pow2 product and residue
Four-state fractal	0	1	Even right half at A
Finned #3	2	3	Ten bounded arithmetic phases

A dependency count includes arithmetic induction lemmas used earlier in the proof graph. It does not count invocations of a lemma afresh, and it is not a count of instructions, scans, time steps, or independent decider algorithms. Zero in the local column means no induction is used to prove the one-step preservation theorem.

All six endpoint statements use the original history definition and exclude every supplied ordinary blank-start halting history. None supplies an invariant or phase certificate as part of the machine input. In the intended natural numbers they imply the stated machines do not halt. These results alone do not give a complete Busy Beaver value or a general soundness theorem for a whole class of deciders.

12.1 Formal project references

The canonical database is `idelta0_machines.mm`. For each name `cycler`, `translated`, `counter`, `bouncer`, `fractal`, or `finned3`, its main labels are

`skelet17-ladder-`*name*`-local`
`skelet17-ladder-`*name*`-history`
`skelet17-ladder-`*name*`-no-blank-halting-history.`

The common prefix is a historical namespace; it does not mean that the six results prove Skelet #17 nonhalting. `ladder_manifest.json` records the actual induction dependencies and file hashes; `ladder_verification.log` records the independent kernel check.

The arithmetic basis is specified in `bb3_metamath_extension/proof_tools/foundation.py`. The division instance and bounded Pow2 definitions are in `skelet17_research/build_binary.py`. The translated, counter, bouncer and fractal local proof generators are respectively `ladder_translated.py`, `ladder_counter.py`, `ladder_bouncer.py`, and `ladder_fractal.py`.

For Finned #3, `ladder_finned3_sweeps.py` gives the phase lemmas and Pow2 multiplication; `finned3_bounds.py` gives the uniform bounds; `ladder_finned3_global.py` assembles the bounded invariant; `ladder_finned3_history.py` constructs the history proof. The independent endpoint checker is `check_finned3_endpoint_scope.py`. It checks the statement's encoding; the Metamath verifier checks the proof itself.

12.2 Reading the actual deductions

With the database loaded in Metamath, for example use

```
show statement skelet17-ladder-finned3-local / full
show proof skelet17-ladder-finned3-local / depth 4
show proof skelet17-ladder-fractal-local / lemmon / renumber
```

The first command displays hypotheses and variable conditions. The second summarizes proof structure; the third expands deductions with line references. Generated syntax and equality nodes account for much of the raw file's length. They are not extra mathematical assumptions. This exposition suppresses those repetitive steps while specifying the invariants, induction instances and logical route to each endpoint.

12.3 Further reading

For broader background on weak arithmetic, see Petr Hájek and Pavel Pudlák, *Metamathematics of First-Order Arithmetic* (Springer, 1993). For the proof language and independent verification approach, see Norman Megill and David A. Wheeler, *Metamath: A Computer Language for Mathematical Proofs*, second edition (2019). The specific machine invariants and dependency counts in this document come from the project files identified above, rather than from those general references.